

Software Defined Ultrasound with GPU Processing

Advanced GPU Short Course

Piotr Jarosik, Marcin Lewandowski, Piotr Karwat

us4us Ltd.

Agenda

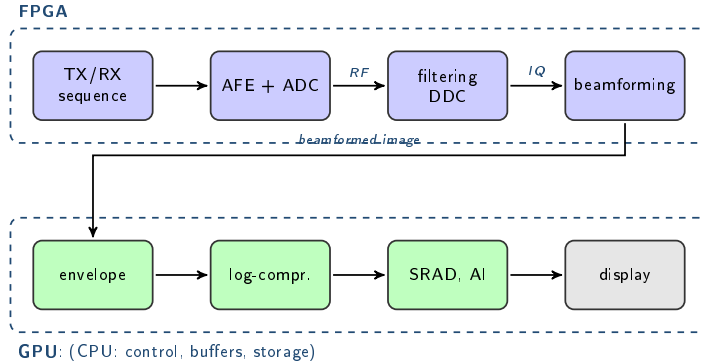
System architecture

Ultrasound

GPU

System architecture

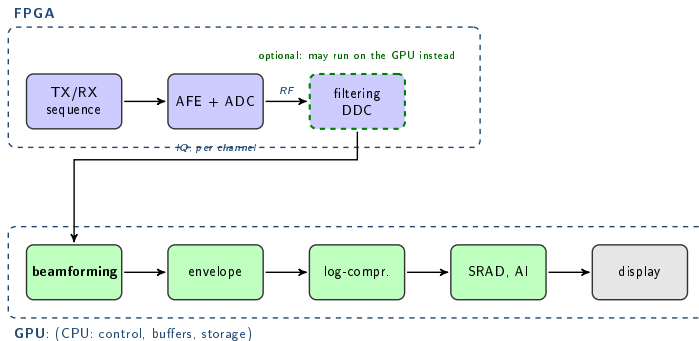
Example: ultrasound imaging system



- **FPGA:** TX/RX sequence execution, RF/IQ acquisition, filtering, beamforming – deterministic, close to the converters.
- **GPU:** everything downstream – flexible, high throughput.

What about the flexibility of RF/IQ data pre-processing and beamforming?

Example: beamforming moved to the GPU



- **the advantage:** the ultrasound imaging becomes *software* – the image reconstruction itself can be changed without touching the firmware (shorter implement → compile → verify loop).
- **the price:** now we need to transfer $n_{tx} \times n_{rx} \times n_{samples}$ data from ultrasound to GPU to reconstruct a single image – orders of magnitude more data.
- **filtering / DDC** may sit on either side: on the AFE it reduces data transfer size, on the GPU it is one more kernel you can improve.

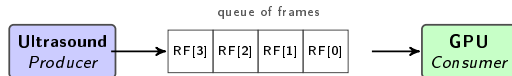
Performance requirements: real-time processing

- **hard real-time**: a missed deadline is a **mission failure** – it may result in patient harm
 - ▶ Example: surgical ultrasound, HIFU
 - ▶ Requirement: the TX/RX sequence must be executed with a constant PRF, and the processing must keep up with the acquisition.
- **soft real-time**: a missed deadline **degrades the quality**, but it is not mission critical
 - ▶ Example: vascular imaging, Color Doppler
 - ▶ Requirement: a dropped or late B-mode frame is acceptable as long as it happens rarely
- **non real-time**: there are **no** requirements regarding frame rate or PRF
 - ▶ Examples: offline reconstruction, research processing

The barrier between the devices

Every boundary between two devices is a **buffer**: a queue of frames, written by the producer and read by the consumer.

- it decouples the two sides: only the *average* rates have to match,
- at least **two elements** are needed, so that both sides can work at the same time,
- deeper queue: more tolerance for jitter, but more **latency**.

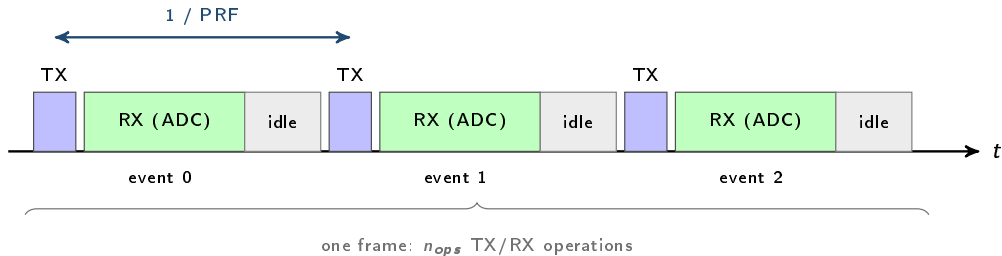


The barrier: what if the consumer is too slow?

- **block the producer** (back-pressure): the ultrasound waits until there is no some room in memory for new data. Nothing is lost, but the PRF follows the processing $\Rightarrow$ possible jitter and lower PRF.
 - ▶ Applicable to: **non real-time** – offline reconstruction, recording, where no frame may be lost and the PRF does not matter.
- **drop frames**: the producer overwrites the oldest element, or skips the newest one.
 - ▶ Applicable to: **soft real-time** – live B-mode or Doppler display, which only needs the newest frame.
- **error on override**: the ultrasound reports an error when it would overwrite an element that is still in use.
 - ▶ Where applicable: **hard real-time** – nothing is lost silently, at the cost of a hard deadline on the consumer.

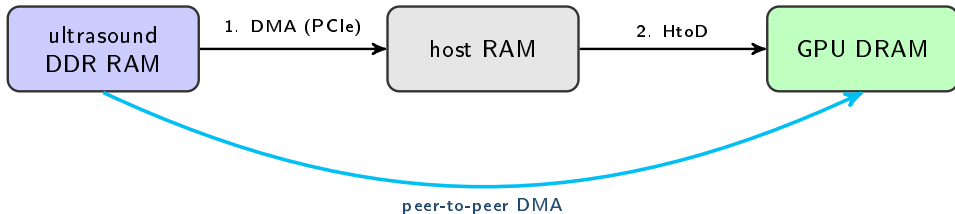
Ultrasound

TX/RX sequence and data acquisition



- the sequence is executed by the **hardware** (FPGA), not by the host: the timing does not depend on the operating system,
- one frame = $n_{ops} \times n_{channels} \times n_{samples}$ samples – this is what has to be transferred every frame.

Data path: DMA to the host or to the GPU



- typically: the data goes from ultrasound to GPU through host RAM,
- **GPUDirect RDMA (NVIDIA) / DMA-BUF, PeerDirect (AMD)**: the RF data is written directly into GPU memory – the host RAM is skipped, the CPU only configures the transfer,
- requires: a device that can do peer-to-peer PCIe (not all GPUs support that), large BAR (host specific), ultrasound driver support.

GPU

Invoking the kernels when new data arrives

The processing is **data-driven**: a new frame arrives, the pipeline is started.

- ultrasound signals a new frame (interrupt, etc.),
- the host **enqueues** the pipeline of kernels,
- the same sequence of kernels is executed for every frame.

How to express and run the **collection of kernels**?

- **CUDA/ROC streams**: the queue of operations (+ events between streams),
- **CUDA/ROC graphs**: the whole pipeline recorded once, launched per frame,
- **Holoscan SDK**: the pipeline as a graph of operations, with a scheduler.

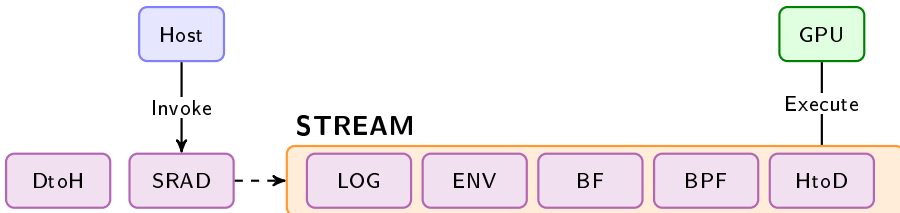
CUDA Stream: introduction

We can run a sequence of GPU kernels to implement the complete e.g. B-mode imaging:

```
// HtoD  
cudaMemcpy(iq_gpu, iq_cpu, iq_bytes, cudaMemcpyHostToDevice);  
bp_filter_kernel    <<<grid, blk>>>(iq_f,  iq_gpu);  
beamform_kernel    <<<grid, blk>>>(hri,   iq_f);  
envelope_kernel    <<<grid, blk>>>(amp,   hri);  
log_compression_kernel<<<grid, blk>>>(bmode, amp);  
srad_kernel        <<<grid, blk>>>(out,   bmode);  
// DtoH  
cudaMemcpy(out_cpu, out, out_bytes, cudaMemcpyDeviceToHost);
```

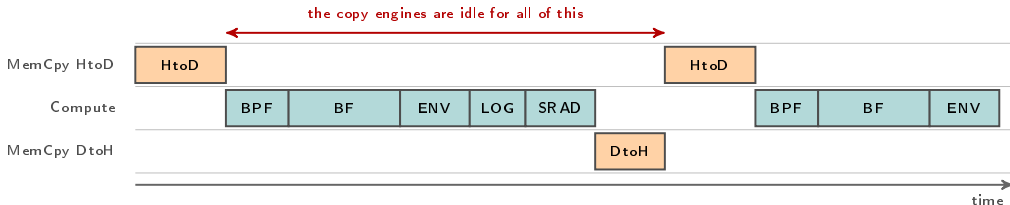
CUDA Streams: example

```
cudaMemcpy(iq_gpu, iq_cpu, iq_bytes, cudaMemcpyHostToDevice); // HtoD
bp_filter_kernel    <<<grid, blk>>>(iq_f,  iq_gpu);    // BPF
beamform_kernel     <<<grid, blk>>>(hri,   iq_f);       // BF
envelope_kernel     <<<grid, blk>>>(amp,   hri);       // ENV
log_compression_kernel<<<grid, blk>>>(bmode, amp);      // LOG
srad_kernel         <<<grid, blk>>>(out,   bmode);     // SRAD
cudaMemcpy(out_cpu, out, out_bytes, cudaMemcpyDeviceToHost); // DtoH
```



CUDA Streams: example

```
cudaMemcpy(iq_gpu, iq_cpu, iq_bytes, cudaMemcpyHostToDevice); // HtoD
bp_filter_kernel    <<<grid, blk>>>(iq_f,  iq_gpu); // BPF
beamform_kernel     <<<grid, blk>>>(hri,   iq_f); // BF
envelope_kernel     <<<grid, blk>>>(amp,   hri); // ENV
log_compression_kernel<<<grid, blk>>>(bmode, amp); // LOG
srاد_kernel        <<<grid, blk>>>(out,   bmode); // SRAD
cudaMemcpy(out_cpu, out, out_bytes, cudaMemcpyDeviceToHost); // DtoH
```



CUDA Streams: multiple streams

Is it possible to run GPU operations in parallel?

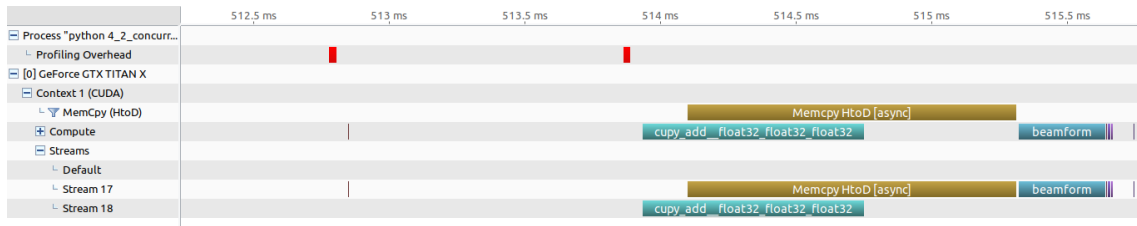
- Generally, it's possible, by using multiple streams.
- In practice: it depends what operations we would like to run in parallel:
 - ▶ executing GPU kernels simultaneously – not always guaranteed (it depends on the GPU occupancy, kernel scheduler implementation, etc.)
 - ▶ GPU kernels and data transfers can be overlapped – data transfers can be handled by a separate GPU component **DMA**, that can run in parallel with SMs

CUDA Streams: overlapping data transfer with kernels

```
cudaStream_t stream_17, stream_18;
cudaStreamCreateWithFlags(&stream_17, cudaStreamNonBlocking);
cudaStreamCreateWithFlags(&stream_18, cudaStreamNonBlocking);

cudaMemcpyAsync(iq_gpu, iq_cpu, iq_bytes,
                cudaMemcpyHostToDevice, stream_17);
bp_filter_kernel    <<<grid, blk, 0, stream_17>>>(iq_f, iq_gpu);
beamform_kernel     <<<grid, blk, 0, stream_17>>>(hri, iq_f);
envelope_kernel     <<<grid, blk, 0, stream_17>>>(amp, hri);
log_compression_kernel<<<grid, blk, 0, stream_17>>>(bmode, amp);
srad_kernel         <<<grid, blk, 0, stream_17>>>(out, bmode);

add                 <<<grid, blk, 0, stream_18>>>(c, a, b);
```



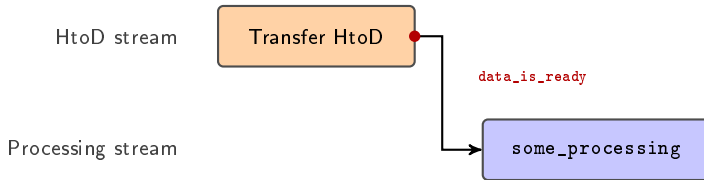
CUDA Streams synchronization: Events

How to make sure, that GPU kernel will not start until the HtoD transfer is finished?

- We can use CUDA **Events**.
- **CUDA event** object represents an event in the CUDA stream execution timeline. An example of such an event is the completion of a GPU kernel, or that the data transfer from host to GPU has finished.

CUDA Streams synchronization: Events

```
cudaEvent_t data_is_ready;  
cudaEventCreate(&data_is_ready);  
  
cudaMemcpyAsync(iq_gpu, iq_cpu, iq_bytes,  
                cudaMemcpyHostToDevice, htod_stream);  
cudaEventRecord(data_is_ready, htod_stream);  
  
cudaStreamWaitEvent(processing_stream, data_is_ready, 0);  
some_processing<<<grid, blk, 0, processing_stream>>>(iq_gpu);
```



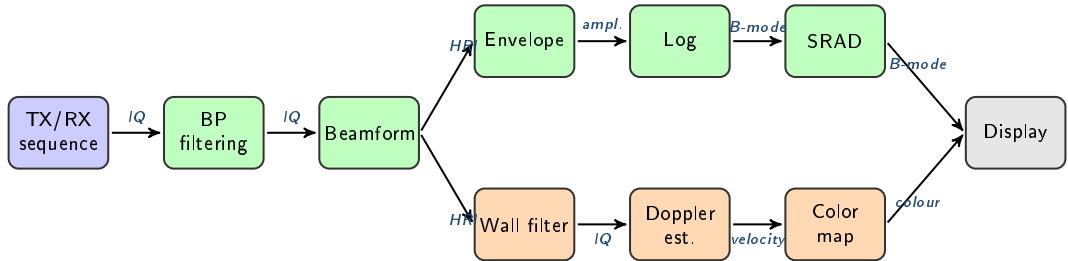
CUDA Streams synchronization: Events

```
cudaEvent_t data_is_ready;  
cudaEventCreate(&data_is_ready);  
cudaMemcpyAsync(iq_gpu, iq_cpu, iq_bytes,  
                cudaMemcpyHostToDevice, htod_stream);  
cudaEventRecord(data_is_ready, htod_stream);  
cudaStreamWaitEvent(processing_stream, data_is_ready, 0);  
some_processing<<<grid, blk, 0, processing_stream>>>(iq_gpu);
```



Graph of operations

It's possible to use CUDA kernels and events to implement a **graph** of operations: a graph $G(V, E)$ with V the **operations** and E the **dependencies** between them (the input/output arrays).

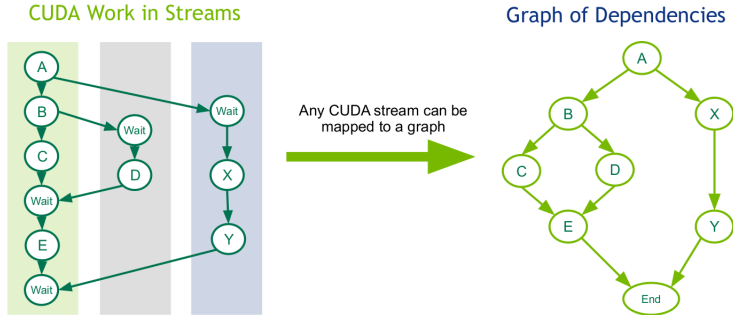


Beamform has two consumers, so the pipeline is no longer a chain – and the two branches are independent, which is exactly what a graph can express and a single stream cannot.

Graph of operations: frameworks

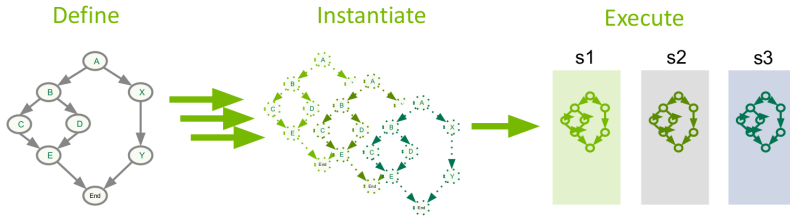
- tensorflow (based on XLA)
- ...
- NVIDIA CUDA Graphs
- NVIDIA CUDA Holoscan SDK
- ARRUS

CUDA Graphs



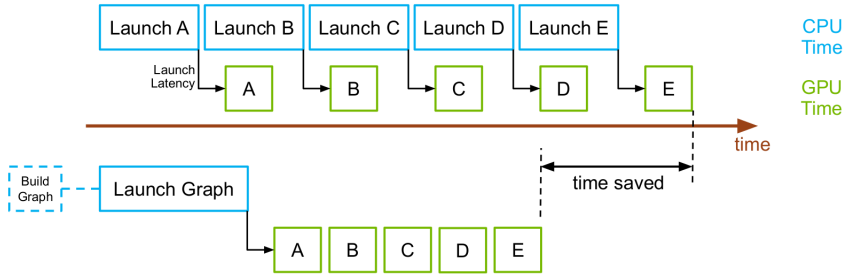
Source: "013 NVIDIA CUDA Graphs", https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

CUDA Graphs: Execution model



Source: "013 NVIDIA CUDA Graphs", https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

CUDA Graphs: kernel invocation overhead reduction



Source: "013 NVIDIA CUDA Graphs", https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

NVIDIA Holoscan SDK

NVIDIA Holoscan is the AI sensor processing platform that combines:

- hardware systems for low-latency sensor and
- network connectivity,
- optimized libraries for data processing and AI,
- ...

NVIDIA Holoscan SDK is a development kit that provides:

- C++ and Python API,
- Built-in, common operators (AI inference, etc.),
- examples, a remote repository of operators, performance tools...

Source: "013 NVIDIA CUDA Graphs", https://www.olcf.ornl.gov/wp-content/uploads/2021/10/013_CUDA_Graphs.pdf

NVIDIA Holoscan SDK: Core concepts

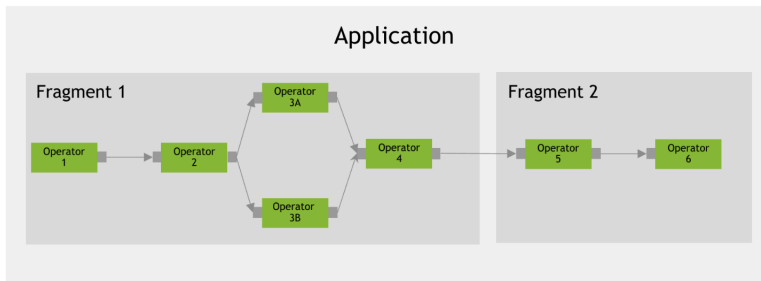
Holoscan SDK implements a custom graph framework: **Graph Execution Framework (GXF)**.

Also provides a native support for:

- GPU Direct RDMA
- TensorRT
- ...

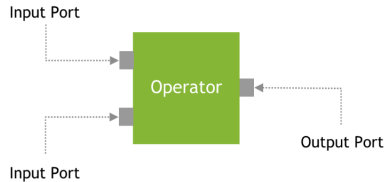
NVIDIA Holoscan SDK: Core concepts

An **Application** is composed of **Fragments**, each of which runs a sub-graph of **Operations**.



NVIDIA Holoscan SDK: Core concepts

Operator is a basic unit of work. An operator receives streaming data at an input port, processes it, and publishes it to one of the output ports.



NVIDIA Holoscan SDK

HoloHub: a central repository to share reusable operators and applications with the Holoscan community.

The screenshot displays the NVIDIA HoloHub website, which serves as a central repository for sharing reusable operators and applications. The header includes the NVIDIA logo, the title "HoloHub - Explore The Holoscan Ecosystem", a search bar, and a GitHub repository link for "nvidia-holoscan/holohub". A navigation menu lists "Home", "Holoscan SDK", "HoloHub", "Holoscan Sensor Bridge", "Support", and "Blog".

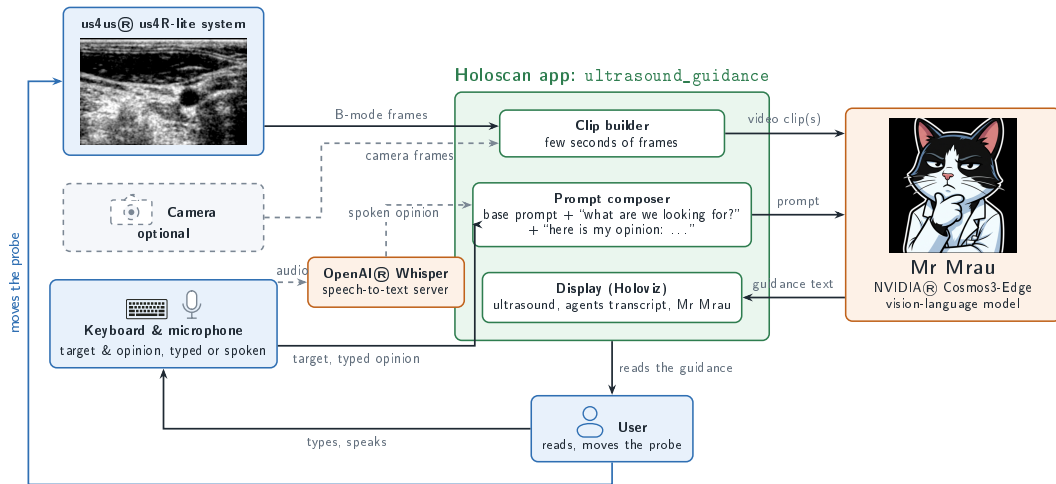
The main content area is titled "Holoscan Applications" and includes a description: "Explore HoloHub's collection of Holoscan applications — reference pipelines showcasing NVIDIA Holoscan across healthcare, computer vision, robotics, streaming, and more. Use these production-ready examples as starting points for your own real-time AI sensor processing workflows in the Holoscan ecosystem." A sidebar on the left lists various application categories: "All (114)", "Camera (13)", "Computer Vision (33)", "Distributed (11)", "Extended Reality (6)", "Healthcare AI (35)", "Holoscan Sensor Bridge (1)", "Medical Imaging (5)", "NVIDIA Video Codec (4)", and "NLP & Conversational AI".

Three application cards are featured:

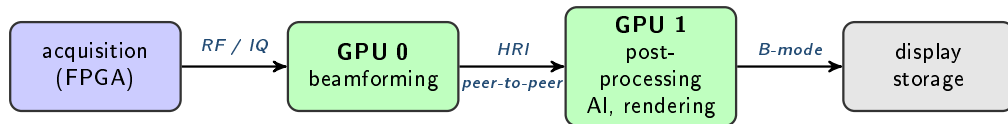
- USRP Spectrum Viewer**: A C++ application that receives RF samples from a USRP radio and ingests CHDR packets through DAQIRL. It is marked as "New" and includes tags for "usrp", "sd", "dplk", and "holoviz".
- Procedural Narrator**: A Python application that displays a replay or V4L2 clinical video with ongoing temporal narration from an OpenAI-compatible multimodal reasener. It includes tags for "generative ai", "multimodal model", "video", and "visualization".
- ROS2 VB1940 (Eagle) Camera on Jetson Thor**: A C++ application that is a port of the VB1940 camera example to NVIDIA Jetson AGX Thor, containing both publisher and subscriber components. It includes tags for "ros2", "vb1940", "eagle", "publisher", "subscriber", "jetson thor", and "mgle".

Source: <https://nvidia-holoscan.github.io/holohub/>

NVIDIA Holoscan SDK Example: Ultrasound Guidance



Example: two GPUs



- one device dedicated to the **fixed-cost** part of the pipeline (beamforming), another one to the **variable-cost** part (AI, rendering, recording),
- the frames are passed between the GPUs directly (peer-to-peer),
- the alternative to two devices: partition a single GPU (see: GPU processing).

Multiple GPUs

- **Peer-to-peer access** (GPU ↔ GPU)

Use: a pipeline split across two devices. *Mechanism:* one device reads or writes the memory of another directly, without staging through the host (`cudaDeviceEnablePeerAccess`, `cudaMemcpyPeerAsync`; HIP: `hipDeviceEnablePeerAccess`, `hipMemcpyPeerAsync`). Carried over NVLink or xGMI where present, over PCIe otherwise.

- **Collective communication libraries** (many GPUs)

Use: one reconstruction distributed over several devices; data-parallel training. *Mechanism:* **NCCL** (NVIDIA) and **RCCL** (AMD) implement broadcast, reduce, all-reduce and all-gather. The library discovers the interconnect topology and selects a transfer schedule accordingly. GPU-aware MPI exposes the same operations.

- **Unified virtual addressing across devices**

Use: code that need not track which device owns a buffer. *Mechanism:* host and all devices share one virtual address space, so a pointer identifies its owner. With peer access enabled, a kernel may dereference a pointer into another device's memory.

Isolating GPU resources: NVIDIA Multi-process service

In a real system more than one *process* wants the GPU: acquisition, beamforming, an AI inference service, the display.

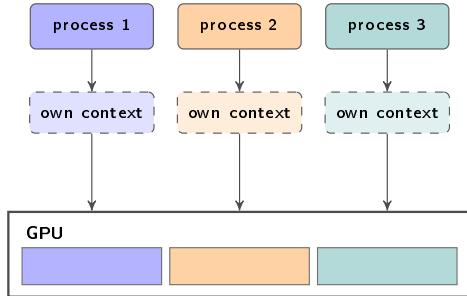
- every process gets its own **CUDA context**,
- the GPU **time-slices** between contexts: at any instant only one context's kernels are executing,
- consequently, four processes each occupying a quarter of the device do *not* saturate it: they execute in turn, and during each turn the majority of the multiprocessors remain idle.

MPS (Multi-Process Service)

Route the work of several processes through a **single** CUDA context, so that their kernels may execute **concurrently** on the same device.

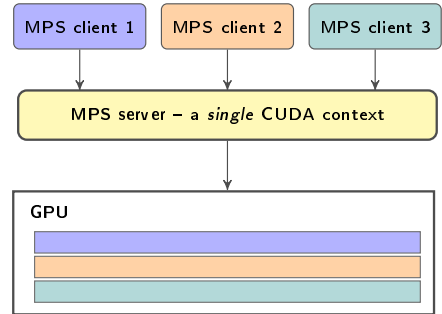
NVIDIA MPS: how it works

without MPS



time-sliced: one context at a time

with MPS



one context: kernels run concurrently

- left: the device switches between *contexts*, so the colours never overlap in time,
- right: a single context, so the multiprocessors carry work from all three clients simultaneously.

NVIDIA MPS: deployment and limitations

A **control daemon** (starts and stops servers), one **server** per user per device (owns the context), a **client runtime** in every CUDA process.

```
# admin: start the daemon (it spawns the server on first use)
export CUDA_VISIBLE_DEVICES=0
nvidia-cuda-mps-control -d          # echo quit / ... to stop it
# per client: cap its share of the GPU before launching it
export CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=25
export CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0=4G
./beamforming_service
```

Provides

- concurrent kernel execution across processes,
- up to 48 clients (16 before Volta),
- per-client multiprocessor and memory limits.

Does not provide

- isolation: the limits are ceilings, not reservations,
- fault containment: a fatal error may terminate the other clients,
- operation without administrative rights for the daemon.

MPS compared with MIG: MPS *shares* one device between processes; MIG *partitions* it in hardware.

A guaranteed allocation requires MIG.